

PROGRAMMERINGSPARM 97b, 11 DEC 1997

Ändrad 1 Dec 11:48

Föreläsning 6

Visserligen har jag tidigare sagt att `char` är en heltalstyp. Men det är inte en slump att typen heter `char`. Den används till att lagra *tecken*. Dvs bokstäver, siffror, interpunktering och andra tecken (eng. *characters*).

En `char` rymmer i en byte, dvs åtta bitar. Ett värde av typen `char` kan skrivas ut både som heltal (med `%d`) och som tecken (med `%c`). I decimal notation är en `char` ett heltal mellan 0 och 255.

Följande program läser in ett tecken med hjälp av `scanf` och `%c`. Sedan skrivs värdet ut som tecken och heltal.

```

1  /*
2  Övnings exempel 9-1, Ping-97.
3  Jan Tångring, Umeå 9/9 1997
4
5  Nyheter:
6      - kommentar
7      - interaktionen väntar på \n
8      - char med scanf -- \n också char!
9      - char som %c och %d
10     - " i sträng_SOURCE_
11     - evig loop -- hur bryta?
12     - 1 som villkor --> 1/0 som logiska värden
13  */
14
15  #include <stdio.h>
16  main()
17  {
18      char ch;
19
20      while(1){
21          scanf("%c", &ch);
22          printf("Tecken: \"%c\", Tal: %d\n", ch, ch);
23      }
24  }
```

Nyheter i programmet, punkt för punkt:

/* Kommentarer */

Programmet inleds med en *kommentar*, inramad med parent `/*` och `*/`.

Kommentarer är något som datorn hoppar över. Människor, som exempelvis Björn och Thomas, vill däremot gärna läsa kommentarer. Kommentarer som berättar vad programmet gör, vem som skrivit programmet, och om det behövs, kommentarer som förklarar hur programmet fungerar.

Kommentarer kan stoppas in nästan var som helst i programmet. Ni kommer att få se fler exempel.

Evig loop, sant/falskt=1/0

Programmet innehåller en *evig loop*. Villkoret i `while`-loopen är av ett nytt slag. Villkoret lyder -- kort och gott -- `1`.

Vad är det för ett märkligt villkor? »`1`«?

Jo -- man brukar tillspetsat säga att »i C är allting heltal«. Detta gäller även logiska värden:

- *Falskt* = `0` -- oftast heltalet, men även flyttalet.
- *Sant* = allt utom `0` -- men helst heltalet `1`

De *villkor* vi tittat på (exvis `i <= 10`) är i själva verket aritmetiska uttryck! Om villkoret är sant, är värdet `1`, om det är falskt är värdet `0`.

Exempelvis är följande tilldelning fullt tillåten:

```
j = (i <= 10);
```

Tilldelningen betyder att `j` sätts till `0` eller `1`. Beroende på om villkoret är sant eller ej.

Vi tar ett exempel till eftersom det hela är så mysco. Dina ögon ljuger inte:

```
i = 5;
j = 16;
k = 10 * (j < 10) + 100 * (i < 10);
```

Variabeln `k` blir `100` eftersom

- $(j < 10) = (16 < 10) = \text{falskt} = 0$
- vilket ger att $10 * (j < 10) = 10 * 0 = 0$
- $(i < 10) = (5 < 10) = \text{sant} = 1$
- vilket ger att $100 * (i < 10) = 100 * 1 = 100$
- Sammantaget $k = 0 + 100 = 100$

Det är alltså inget fel på `while`-loopen på rad 20. Eftersom `1` betyder *sant*, kommer loopvillkoret alltid vara sant, och loopen kommer att fortsätta snurra i all evighet.

Fast bara i princip. I praktiken kan användaren bryta programmet. Till exempel genom att starta om datorn. Eller -- som jag gjorde -- genom att trycka `[Control]+[c]`.

Det är inte säkert att `[Control]+[c]` fungerar på ert system. Thomas och Björn vet hur man bryter program på ert system.

WARNING: Tilldelning har ett värde

Tilldelningssatsen är ett aritmetiskt uttryck! I stället för

```
i = 1;
j = 1;
```

skriver C-programmerare ofta

```
i = j = 1;
```

eller (vilket är samma sak):

```
i = (j = 1);
```

Det är två tilldelningar vi ser. Den första tilldelningen är

```
j = 1.
```

Och tilldelning är en sats med ett värde. Värdet är detsamma som variabeln får. Vi kan använda detta värde i en ny tilldelning!

Den andra tilldelningen är sålunda `i = 1`.

Vi ska senare få se en ganska vanlig användning av denna märkliga möjlighet (i [kursboken](#) finns användningen som exempel redan på sidan 29).

Värdet av tilldelningssatsen är samma värde som vänstersidans variabel får.

Detta behöver inte vara detsamma som värdet av högersidan eftersom högersidan kan omvandlas innan tilldelning sker. Detta sker till exempel om en `float` tilldelas värdet av en `int`.

C accepterar glatt

```
while(i=j)
om du råkar skriva det istället för
while(i==j).
```

Detta eftersom tilldelningssatsen har ett värde. Och eftersom nästan alla tänkbara värden duger som logiska värden.

char som %c och %d

På rad 22 ovan skrivs `ch` ut på två olika sätt.

Variabeln `ch` är alltid ett heltal. Men om vi skriver ut värdet med specifikationen `%c` skriver datorn ut ett tecken istället för ett tal.

Tecken och heltal är kopplade till varandra på olika sätt i olika system. Mellan 0 och 127 är det dock ganska standardiserat. Den standarden har funnits i många år och heter [ASCII](#) (se [Kurslitteratur](#) sidan 36).

Våra å, ä och ö finns dock inte bland de 128 ASCII-tecknen. Därför finns utökade versioner av ASCII som inkluderar dessa och andra försummade tecken.

Dumt nog finns *flera olika* utökade ASCII-versioner. Era PC, min Macintosh, och den UNIX ni lär er, använder olika koder för å, ä och ö. Bara för att ta några exempel.

Detta har lett till en otrolig mängd strul. Till exempel när elektronisk post (snabelpost) ska skickas mellan olika system.

Jag lider själv av strulet i skrivande stund. För att försäkra mig om att denna text är läsbar på alla

maskiner, byter jag ut tecknen mot den standard som används på WWW. Exempelvis skriver jag *ö* som *ö*;
WWW-standarden utnyttjar endast de klassiska ASCII-värdena mellan noll och 127.

Åter till programmet

Så här ser programmet ut när man kör det på mitt system.

Det första vi ser är användarens inmatning. Därefter följer datorns »analys«. Datorn plockar sönder inmatningen och skriver tecknen, ett i taget. Både som tecken och som tal.

```
1 abc
2 Tecken: "a", Tal: 97
3 Tecken: "b", Tal: 98
4 Tecken: "c", Tal: 99
5 Tecken: "_SOURCE_"
6 ", Tal: 10_SOURCE_"
7 123
8 Tecken: "1", Tal: 49
9 Tecken: "2", Tal: 50
10 Tecken: "3", Tal: 51
11 Tecken: "_SOURCE_"
12 ", Tal: 10_SOURCE_"
13 ^C
```

Användarens inmatningar blandas med datorns utskrifter när programmet körs.

I programmet ser det ut som om användare och dator turas om. Det ser ut som om datorn läser ett tecken i taget, och skriver sin analys för det tecknet.

Men i utskriften skriver användaren flera tecken innan datorn reagerar!

Det beror på att datorn läser tecknen stötvis *en rad i taget*.

Datorn väntar tills användaren tryckt på returtangenten. Då läser datorn in hela raden, och mellanlagrar alla tecknen i en *buffert*. Därefter betas tecknen i bufferten av -- ett efter ett -- varje gång *scanf* anropas.

Efter ett tag är tecknen i bufferten slut. Då stannar datorn upp, och väntar på en ny textrad från användaren.

Så går det alltid till när *scanf* används. Inte bara vid inläsning av *char*.

Resultatet på skärmen kan bli förvirrande om användaren kommer ur fas med datorn.

Utskrift av " , med mera

Vissa tecken är inte triviala för *printf* att skriva ut. Tecknet " till exempel, används till att rama in strängen som ska skrivas ut. Hur ska " skrivas utan att datorn tror att strängen avslutas? Svaret finns på rad 22 i programmet.

Att tekniken fungerar ser ni i utskriften.

Följande lilla tabell visar hur tecknet " och andra besvärliga tecken anges i *printf*:

- " anges med \"
- \ anges med \\
- % anges med %%

Teckenkonstanter

Inuti ett C-program kan man skriva värden av typen `char` dels som heltal, dels som tecken. När man skriver dem som tecken, ramar man in med apostrofer (`'`).

Följande två tilldelningar betyder exakt samma sak:

- `ch = 65;`
- `ch = 'A';`

Man använder `\` för att komma åt besvärliga tecken:

- `'\n'` = *ny rad*
- `'\"'` = `"`
- `'\''` = `'`
- `'\\'` = `\`

Tänker du använda några av ovanstående tecken många gånger i ditt program, kan det vara en god idé att definiera dem som makron, exempelvis:

```
#define BACKSLASH '\\'  
#define NEWLINE '\n'
```

Ny rad är ett tecken

Även *ny rad* representeras av ett tecken. Tecknet har värdet `10 = '\n'`.

När `scanf` har som uppdrag att läsa in tecken till en variabel, läser `scanf` också in nyradstecknet.

När `printf` skriver ut nyradstecknet, blir det en radframmatning. Det är därför utskriften bryts upp i rad 5 och 6.

Nästa lektion

